

AUTHENTICATION HANDBOOK

≈ (JWT + OAuth 2.0) ≈

1. Authentication Fundamentals

Authentication is about verifying who you are. **Authorization** is about what you are allowed to do.

1.1 Authentication vs Authorization



Authentication

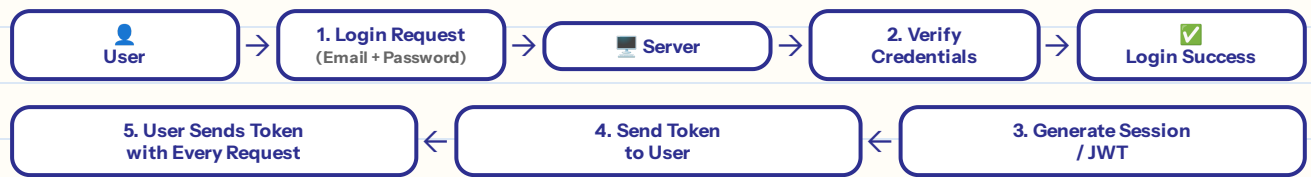
Verifies the identity of a user.
Example: Login with email & password.



Authorization

Determines what a user can access.
Example: Accessing admin dashboard.

1.2 Basic Login Flow



1.3 Sessions vs Tokens

	Session-Based (Auth)	Token-Based (JWT)
Where data is stored	Server	Client (Token)
Stored on	Cookie (Session ID)	LocalStorage / Cookie / Memory
Scalability	Harder (needs session store)	Easier (Stateless)
Revocation	Easy (invalidate session)	Harder (need blacklist)
Best for	Traditional Web Apps	APIs, SPAs, Mobile Apps

1.4 Cookies (Quick Overview)

- ★ Small pieces of data stored in browser.
- ★ Sent automatically with every request to the domain.
- ★ Can store Session ID or JWT.

HttpOnly → Not accessible via JavaScript (protects from XSS)
Secure → Sent only over HTTPS
SameSite → Controls CSRF attacks (Lax / Strict / None)

1.5 Common HTTP Status Codes

401 Unauthorized

Authentication is required or failed (invalid/missing token or credentials).

403 Forbidden

Authenticated, but not allowed to access the resource.

★ 1.6 Key Takeaway

Authentication answers the question: **"WHO ARE YOU?"**
 Authorization answers the question: **"WHAT CAN YOU DO?"**

AUTHENTICATION HANDBOOK

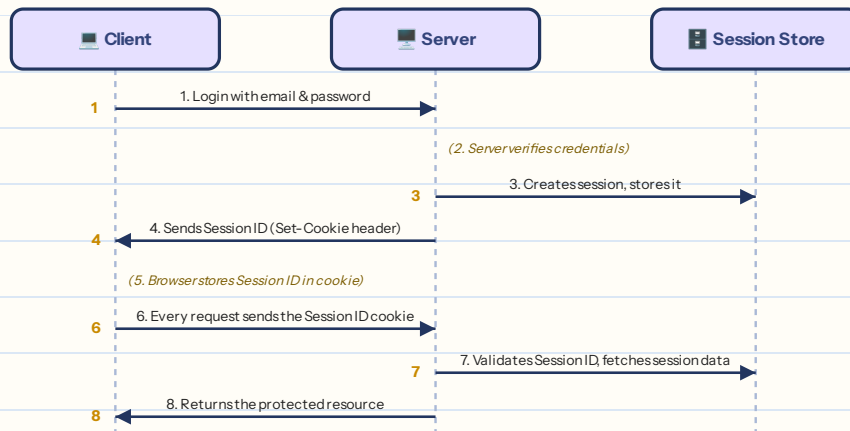
~ (JWT + OAuth 2.0) ~

PAGE 2

2. Session-Based Authentication

In session-based auth, the server stores user session information. The client only holds a **Session ID** (in cookie).

2.1 How It Works (Flow)



2.3 Session Store (Where to Store?)

- ★ **Database** → Easy but slower
- ★ **Redis** → Fast, scalable, widely used
- ★ **Memcached** → Very fast (key-value store)
- ★ **In-Memory** → Good for small apps (not scalable)
- ★ **File / Others** → Not recommended for production

2.5 When to Use Sessions?

- ★ Traditional Web Applications
- ★ When you need to logout from all devices
- ★ When immediate session invalidation is required
- ★ When sensitive data should not travel to client
- ★ When you can maintain a server-side store

2.4 Pros and Cons

Advantages	Disadvantages
✓ Simple to implement	✗ Not stateless (needs server storage)
✓ Easy to revoke / invalidate session	✗ Harder to scale (requires shared store)
✓ More secure (Session ID is small)	✗ Extra load on server / store
✓ Works well with traditional web apps	✗ Not ideal for mobile / APIs
✓ Can store more data on server	✗ More server memory usage

2.6 Session vs Token (Quick Recap)

Feature	Session-Based	Token-Based (JWT)
Where data is stored	Server (Session Store)	Client (Token)
What the client holds	Session ID (in Cookie)	JWT (Full token)
Scalability	Harder (needs shared store)	Easier (Stateless)
Revocation	Easy (Delete session)	Harder (Blacklist / Expiry)
Best for	Web Apps	APIs, SPAs, Mobile Apps

Remember

Sessions keep data on the server.
Tokens keep data on the client.

Server VS Client

3. JWT (JSON Web Token) Fundamentals

3.1 JWT Structure



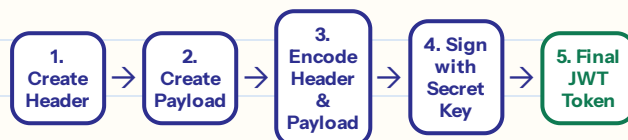
3.4 Signature

3.3 Payload / Claims (Example)

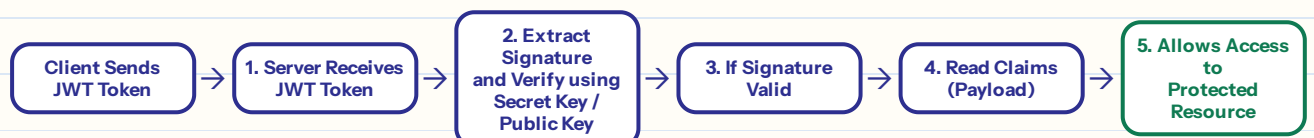
3.5 Encoding vs Encryption

Aspect	Encoding (Base64)	Encryption
Purpose	Make data readable in JSON format	Hide data (keep secret)
Can be decoded?	Yes (Anyone)	Not Encryption
JWT uses?	Encoding	No
Safe for secrets?	No	Yes
Example	Base64UrlEncode	AES, RSA, etc.

3.6 JWT Creation Flow



3.7 Verification Flow (On Server)



p. 3 1 auth notes

AUTHENTICATION HANDBOOK

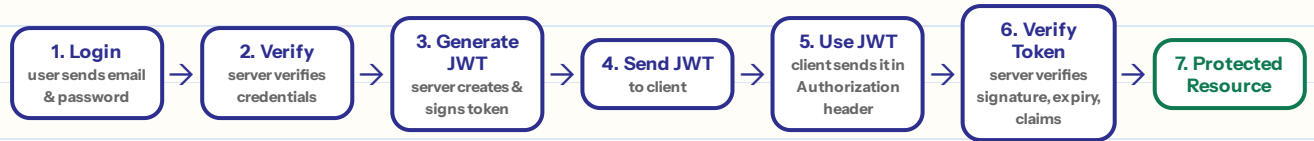
~ (JWT + OAuth 2.0) ~

PAGE 4

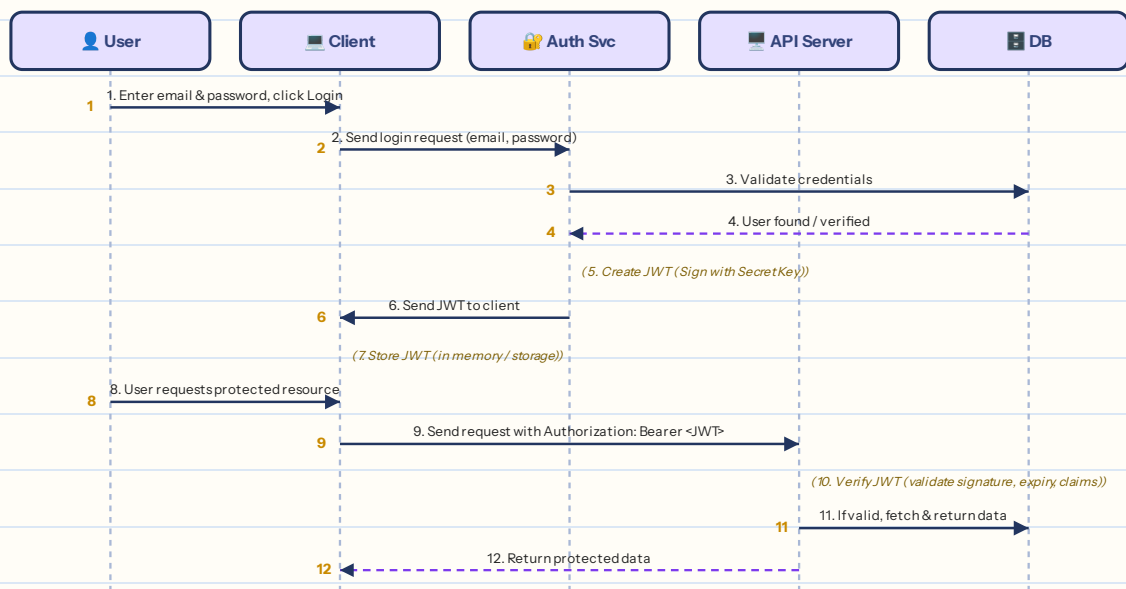
4. JWT Authentication Flow

JWT is stateless. Once issued, the token is used for every request until it expires.

4.1 End-to-End Flow (High Level)



4.2 Detailed Sequence Diagram



4.3 What Happens in Each Step?

Step	What Happens	Who Does It?
1	User submits login form.	User
2	Server checks email & password in DB.	Auth Service
3	If valid, server creates JWT (header.payload.signature).	Auth Service
4	JWT is sent back to client.	Auth Service
5	Client stores JWT and sends in Authorization header.	Client
6	Server (API) verifies JWT on every request.	API Server
7	If valid → access granted and data returned.	API Server

Authorization Header Example

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
IzR5cCI6IkpXVCJ9...

4.4 Notes

JWT is stateless · No session store needed on server · Server only verifies the token · Token contains all required claims · If token is expired/invalid → 401 Unauthorized · If token is valid but not allowed → 403 Forbidden.

AUTHENTICATION HANDBOOK

≈ (JWT + OAuth 2.0) ≈

PAGE 5

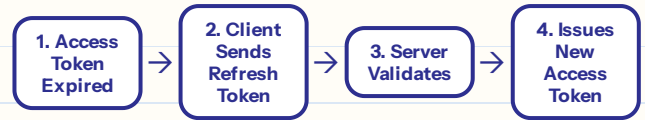
5. Access Token + Refresh Token

Using two tokens improves security and user experience.

5.1 Why Two Tokens?

- ★ **Access Token** → Short lived (5–15 min). Used to access APIs.
- ★ **Refresh Token** → Long lived (Days/Weeks). Used only to get new access token.
- ★ Even if Access Token is leaked, damage is limited (short expiry). Refresh Token stays secure (HttpOnly cookie).

5.2 Refresh Flow



Invalid / Expired / Revoked Refresh Token → 401 Unauthorized

5.3 Token Expiry Strategy (Example)

Token	Expiry
Access Token	5 – 15 minutes
Refresh Token	7 – 30 days
Refresh Token Rotation	Yes (Recommended)
Reuse Detection	Yes (For Security)

5.4 Refresh Token Rotation

User logs in → Use R1 to get new tokens (Access A2 + Refresh R2) → Use R2 to get new tokens (A3+R3) → ...

Old Refresh Token (R1, R2, ...) becomes invalid after it is used once.

5.5 Reuse Detection (Important)

If an old Refresh Token is used again, it means the token is stolen. In this case:

- ① Invalidate that Refresh Token.
 - ② Invalidate all tokens issued after it.
 - ③ Ask user to login again.
- Protects from token theft / replay attack.

5.6 Logout & Revocation

User Logout

Delete Refresh Token from server-side store. Client deletes Access Token from storage. User needs to login again.

Admin / Server Revocation

Revoke Refresh Token (for suspicious activity). Blacklist Access Token (jti) if required. Force logout from all devices (optional).

5.7 Access Token vs Refresh Token

Feature	Access Token	Refresh Token
Purpose	Access protected resources (APIs)	Get new access token
Lifetime	Short (5–15 min)	Long (Days/Weeks)
Sent In	Authorization header (Bearer Token)	HttpOnly Cookie or Secure Storage
Risk if Leaked	Limited (short expiry)	High (can issue new access token)
Storage	Memory / LocalStorage (or Header use)	HttpOnly Secure Cookie (Recommended)
Rotation	No (New one issued often)	Yes (Rotate on every use)
Revocation	Blacklist (jti) if needed	Remove from store / Can be canceled

AUTHENTICATION HANDBOOK

≈ (JWT + OAuth 2.0) ≈

PAGE 6

6. Where Should You Store JWT?

Storing JWT securely is critical. Each option has trade-offs.

6.1 Storage Options Comparison

Storage Option	Accessible by JS?	XSS Risk	CSRF Risk	Persistence	Best For
1. HttpOnly Cookie	No	Low (Protected)	Yes (if no protection)	Configurable (Expires)	Web apps · Most secure for tokens
2. Memory (React State)	Yes	High (Vulnerable to XSS)	Low	None (Lost on refresh)	SPA Apps · Short lived sessions
3. LocalStorage	Yes	High (Vulnerable to XSS)	Low	Long Term (Persists)	Not Recommended for JWT · High XSS Risk
4. SessionStorage	Yes	High (Vulnerable to XSS)	Low	Until Tab Closes	Slightly better than LocalStorage · Still not ideal

✓ 6.2 Recommendation

Best Practice: Store JWT (Refresh Token) in HttpOnly, Secure, SameSite cookies.
Store Access Token in memory (not localStorage) for SPAs if possible.
Always use HTTPS.

🍪 6.3 Important Cookie Flags

HttpOnly → Not accessible via JavaScript (Prevents XSS theft)
Secure → Sent only over HTTPS
SameSite → Controls CSRF attacks (Strict/Lax/None)
Path → Limits where cookie is sent
Max-Age → Defines cookie lifetime (in seconds)

✗ 6.4 Common Security Mistakes

- ✗ Storing JWT in localStorage.
- ✗ Not using HttpOnly cookies for refresh token.
- ✗ Not using HTTPS (token can be sniffed).
- ✗ Not validating token expiration.
- ✗ Not using SameSite cookies (CSRF risk).

🔑 6.5 Quick Decision Guide

Use HttpOnly Cookie when: you want long lived refresh token · you want better protection from XSS · you use cookie-based auth (traditional web apps)

Use Memory (Not localStorage) when: you want short lived access token · you are building an SPA

AUTHENTICATION HANDBOOK

≈ (JWT + OAuth 2.0) ≈

PAGE 7

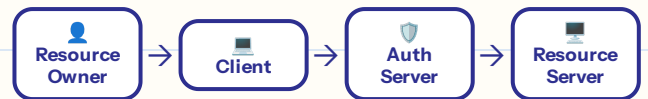
7. OAuth 2.0 Fundamentals

OAuth 2.0 is an **authorization** framework, not an authentication protocol. It allows a user to grant limited access to their resources without sharing credentials.

7.1 Key Roles in OAuth 2.0

 1. Resource Owner (End User)	The user who owns the data and grants permission.
 2. Client (Application)	The application that requests access to user's resources.
 3. Authorization Server (AS)	The server that authenticates the user and issues access tokens to the client.
 4. Resource Server (API Server)	The server that hosts the user's protected resources.

7.2 How OAuth 2.0 Works (High Level)



- ① User tries to access a resource via Client.
- ② Client redirects user to Authorization Server to login and grant permission (with scopes).
- ③ User authenticates and consents.
- ④ Authorization Server issues an Access Token to the Client.
- ⑤ Client uses the Access Token to access the Resource Server.
- ⑥ Resource Server validates the token and returns the requested data.

7.3 Important Concepts

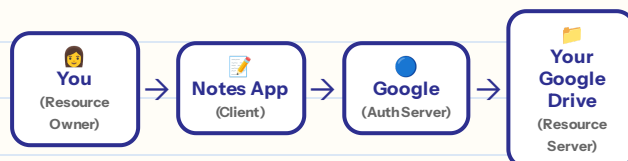
- ★ **Scope** → Defines the level of access requested (e.g., read:profile, write:email).
- ★ **Consent** → User gives permission to the client.
- ★ **Access Token** → Short lived token used to access APIs.
- ★ **Refresh Token** → (Optional) used to get new access token when the old one expires.
- ★ **Grant Type** → The flow used to obtain the token.

7.4 OAuth 2.0 is NOT Authentication

OAuth 2.0 = Authorization (Delegation) — It allows a client to access resources on behalf of the user with user's permission.

Authentication = Proving Identity — It verifies who the user is (e.g., using username and password).

7.5 Example: Real World Analogy



You (user) allow the Notes App to access your Google Drive files through Google (Auth Server) — without sharing your password.

7.6 OAuth 2.0 vs Other Methods

Feature	OAuth 2.0	JWT	Session
Purpose	Authorization (Delegated Access)	Authentication (Token based)	Authentication (Server session)
Stores User Session?	No	No	Yes (on server)
Token Type	Access Token (+Refresh Token)	Access Token (JWT)	Session ID (in cookie)
Stateless?	Yes	Yes	No
Main Use	Allow 3rd party apps access user data	Secure API auth (self-contained)	Web apps with server managed sessions

AUTHENTICATION HANDBOOK

≈ (JWT + OAuth 2.0) ≈

PAGE 8

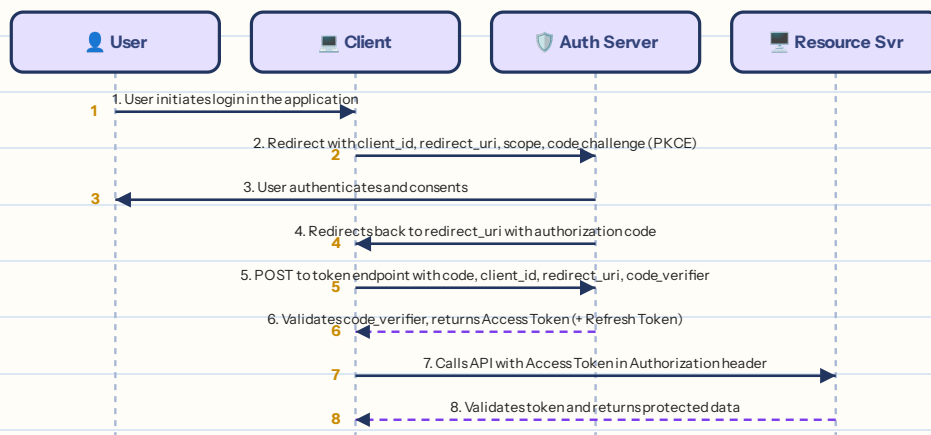
8. OAuth 2.0 Authorization Code Flow (With PKCE)

Authorization Code Flow is the most secure OAuth 2.0 flow for web and mobile apps. PKCE adds extra protection for public clients.

8.1 Actors Involved



8.2 Authorization Code Flow (With PKCE) – Step by Step



What is PKCE?

PKCE (Proof Key for Code Exchange) prevents authorization code interception attacks.

Flow: Client creates a random code_verifier and sends its hash (code_challenge) to the server. While exchanging the code, client sends the original code_verifier. Server verifies both issue tokens only if they match.

8.3 Advantages

- ★ Most secure flow for web and mobile apps.
- ★ Authorization code is short lived.
- ★ PKCE protects against code interception.
- ★ Access token is never exposed in browser URL.

8.4 When to Use?

- ★ Web applications
- ★ Mobile applications
- ★ Applications that can securely store client secret (Confidential clients)

AUTHENTICATION HANDBOOK

≈ (JWT + OAuth 2.0) ≈

PAGE 9

9. OAuth vs JWT vs Sessions

They solve different problems. Choose the right tool for the right job.

Feature	OAuth 2.0	JWT	Sessions
What it is	Authorization framework (Delegated Access)	Token format (For Authentication)	Server-side session management
Main Purpose	Allow a client to access resources on behalf of a user	Prove the identity of the user and carry claims	Maintain user state on the server
Uses Tokens?	Yes (Access Token + Refresh Token)	Yes (Access Token)	No (Uses Session ID in Cookie)
Where is Data Stored?	Authorization Server (+ Client)	Inside the token (Stateless)	On Server (Session Store)
State	Stateful (Auth Server) but supports Stateless	Stateless (Self-contained)	Stateful (Server-side)
Scalability	High (Designed for scale)	Very High (No server state needed)	Low to Medium (Session store needed)
Revocation	Possible (Token revocation, introspection)	Hard (Need blacklist or short expiry)	Easy (Delete session from server)
Best For	3rd party access, SSO, API access delegation	Microservices, Mobile apps, SPAs, Stateless APIs	Traditional web apps, internal systems
Delivered By	Authorization Server (e.g., Google, Auth0)	Your Application / Auth Service	Your Application (Server)

9.1 Common Misconceptions (Clear it!)

- ✗ OAuth 2.0 is NOT an authentication protocol. It is an authorization framework.
- ✗ JWT is NOT OAuth. It is just a token format.
- ✗ OAuth 2.0 can use JWT as Access Token, but it can also use opaque tokens.
- ✓ Sessions can be used with OAuth 2.0 (e.g., in server-side web apps).
- ✓ JWT can be used without OAuth 2.0 (for first-party authentication).

9.2 When to Use What?

- Use OAuth 2.0 When:** you want to allow access to user data by another app · you are building login with Google/Facebook/3rd party apps.
- Use JWT When:** you need stateless authentication for APIs · you are building microservices/mobile/SPA apps.
- Use Sessions When:** you are building a traditional web application · you want easy logout and server-side control.

9.3 Quick Comparison Diagram



AUTHENTICATION HANDBOOK

≈ (JWT + OAuth 2.0) ≈

PAGE 10

10. Security Best Practices

Follow these best practices to keep your authentication secure and robust.



1. Use HTTPS

Always use HTTPS to protect tokens and data in transit. Never send tokens over HTTP.



2. Short Lived Access Token

Keep Access Tokens short lived (5-15 min). Use Refresh Token for long term access.



3. Secure Storage

Store tokens in HttpOnly, Secure, SameSite cookies. Avoid storing tokens in LocalStorage/SessionStorage.



4. Validate Tokens

Always verify token signature and expiry. Validate issuer (iss), audience (aud) and other claims. Reject invalid or tampered tokens.



5. Refresh Token Security

Rotate Refresh Token after every use. Revoke old tokens immediately. Set proper expiry for Refresh Token.



6. Least Privilege

Grant only the permissions that are required. Follow the principle of least privilege.



7. Handle Logout Properly

Clear tokens from client and server. Invalidate Refresh Token on logout. Destroy session (if using sessions).



8. Protect Against CSRF

Use SameSite cookies. Use CSRF tokens for state changing operations.



9. Monitor & Log

Log login/logout attempts. Monitor suspicious activities. Alert on unusual behavior.



10. Keep Secrets Safe

Never expose your client secret. Store secrets in environment variables.

10.1 Token Security Checklist

- ☒ Use HTTPS everywhere.
- ☒ Access Token is short lived (5-15 min).
- ☒ Refresh Token is long lived and rotated.
- ☒ Tokens are stored in HttpOnly, Secure, SameSite cookies.
- ☒ Tokens are not stored in LocalStorage / SessionStorage.
- ☒ Token signature and expiry are validated.
- ☒ Implement proper logout and token revocation.
- ☒ Protect against CSRF attacks.
- ☒ Use least privilege when granting scopes/permissions.
- ☒ Monitor and log authentication events.
- ☒ Keep secrets safe and never commit them.

10.2 Common Security Threats

Threat	How to Prevent
Token Theft (XSS)	Use HttpOnly cookies, Content Security Policy (CSP), input validation.
Replay Attack	Use short lived tokens, token rotation, unique jti claim.
Token Tampering	Validate signature, use strong algorithms (HS256 / RS256 / ES256).
CSRF Attack	Use SameSite cookies, CSRF tokens.
Brute Force Login	Rate limiting, account lockout, CAPTCHA.
Exposed Secrets	Use environment variables, secret management tools.
Expired Token Usage	Validate expiry and return proper error (401 Unauthorized).

10.3 Quick Tips

Security is not optional, it is essential. A small mistake in authentication can lead to big breaches.

10.4 Tools You Can Use

J.T Debugger → jwt.io · **OAuth Playground** → oauth.net / Auth0 Playground

11. Common Interview Questions

- **1. What is the difference between Authentication and Authorization?**
Authentication verifies who you are. Authorization decides what you can access.
- **3. What are the parts of a JWT?**
Header . Payload . Signature (Encoded and separated by dots).
- **5. What is the difference between OAuth 2.0 and JWT?**
OAuth 2.0 is an authorization framework. JWT is a token format used inside OAuth 2.0.
- **7. What is PKCE in OAuth 2.0?**
PKCE (Proof Key for Code Exchange) is an extension used to prevent authorization code interception attacks.
- **9. Where should JWT be stored?**
Best: HttpOnly, Secure, SameSite cookies. Avoid: localStorage and sessionStorage.
- **2. What is JWT?**
JWT (JSON Web Token) is a compact, URL-safe token used to securely transmit information between parties.
- **4. What is the use of Refresh Token?**
Refresh Token is used to get a new Access Token when the old one expires.
- **6. What is scope in OAuth 2.0?**
Scope defines the level of access the client is requesting (e.g., read:profile, write:email).
- **8. How is JWT different from Sessions?**
JWT is stateless and stored on client. Sessions are stateful and stored on server.
- **10. How do you revoke a JWT?**
JWT cannot be revoked directly. Use short expiry + Refresh Token + blacklist.

14. Token Flow Summary

 Authorization Code Grant	Most secure. For web apps, PKCE.
 Implicit Grant	For SPAs (legacy). Token in URL fragment.
 Resource Owner Password Grant	User gives username/password to app directly.
 Client Credentials Grant	For machine-to-machine (no user involved).
 Refresh Token Grant	Used to get new Access Token when old one expires.



- OAuth 2.0 = Authorization Framework
- JWT = Token Format
- Sessions = Server State

AUTHENTICATION HANDBOOK

≈ (JWT + OAuth 2.0) ≈

PAGE 12

12. Real World Examples

Let's see how popular platforms use OAuth 2.0 and JWT.

Platform / Service	Uses OAuth 2.0?	Uses JWT?	How They Use It
Google	Yes	Yes	OAuth 2.0 for login with Google. JWT (ID Token) to identify the user.
Facebook	Yes	Yes	OAuth 2.0 for Facebook Login. JWT to securely send user info to app.
GitHub	Yes	Sometimes	OAuth 2.0 for third-party apps. JWT for GitHub Apps (identification).
Twitter (X)	Yes	Yes	OAuth 2.0 for user login. JWT for access tokens in API calls.
Microsoft (Azure AD)	Yes	Yes	OAuth 2.0 / OpenID Connect for login. JWT (ID Token) for user identity.
Stripe	Yes	No	OAuth 2.0 for API authentication. Uses API keys & tokens (not JWT).
Your App	Yes	Yes	OAuth 2.0 for login (Google, Facebook, etc.) JWT for managing sessions and APIs.

12.1 Design Decision Guide

Use OAuth 2.0 when...	Use JWT when...
You want users to login with Google, Facebook, etc.	You need to identify the user after login.
You are giving third-party access to your API (with scopes).	You want stateless and fast authentication.
You need delegated access (with scopes).	You are building your own auth system.
You need a standards-based and secure flow.	You need to send claims between services securely.

12.2 Summary Cheat Sheet

OAuth 2.0	JWT
✓ Authorization Framework	✓ Token Format
✓ Delegated Access	✓ Authentication
✓ Uses Access Token	✓ Contains Claims
✓ Multiple Flows	✓ Self-contained
✓ Stateful (AS stores info)	✓ Stateless
✓ For Login with 3rd Party (Google, FB, etc.)	✓ For your own API and services

12.3 OAuth 2.0 vs JWT vs Sessions (At a Glance)

Feature	OAuth 2.0	JWT	Sessions
Type	Authorization Framework	Token Format	Server-side State
Purpose	Allow access to resources on behalf of user	Prove identity & carry claims	Maintain user state
Token / ID	Access Token (and Refresh Token)	JWT	Session ID (in Cookie)
Stored Where?	Auth Server	Client (self-contained)	Server (Session Store)
State	Stateful (AS keeps info)	Stateless	Stateful
Best For	3rd Party Login, API Access Delegation	APIs, Microservices, Mobile Apps	Traditional Web Apps
Revocation	Easy (revoke token)	Hard (use short expiry + blacklist)	Easy (delete session)

Notes compiled from a 12-part Authentication Handbook series (JWT + OAuth 2.0) — @abhi_techhub